

Actividad 2: Implementación Física del Robot Autónomo

Solución al Problema de Transmisión de Video en Tiempo Real
mediante Streaming UDP Personalizado

Proyecto Autónoma

6 de julio de 2026

1. Introducción y Contexto del Problema

Durante la transición de la implementación simulada (Gazebo) a la implementación física sobre el robot TurtleBot3, se identificó una limitación crítica en la cadena de transmisión de video que imposibilitaba la detección de objetos en tiempo real.

El sistema original dependía del paquete `usb_cam` de ROS 2 para capturar la imagen de la cámara USB montada sobre la Raspberry Pi del TurtleBot3 y publicarla directamente al tópico `/camera/image_raw`. Este enfoque, que funcionaba perfectamente en simulación, presentó tres problemas simultáneos en el entorno físico:

1. **Corrupción del driver UVC:** El módulo del kernel Linux `uvcvideo` bloqueaba repetidamente el dispositivo `/dev/video0`, generando errores del tipo `Unable to start stream` incluso cuando la cámara era detectada correctamente por el sistema operativo.
2. **Limitaciones de ancho de banda:** El transporte nativo de imágenes de ROS 2 sobre la red Wi-Fi de área local resultaba en pérdida severa de paquetes UDP y fragmentación de mensajes. A resoluciones superiores a 160×120 píxeles el flujo de video se interrumpía completamente, y a dicha resolución el modelo YOLOv8 no podía identificar objetos con precisión aceptable.
3. **Conflicto de backends en OpenCV:** En la Raspberry Pi, OpenCV intentaba usar GStreamer como backend para captura de video (`cv2.VideoCapture(0)`), fallando silenciosamente al no tener un pipeline GStreamer válido configurado, y generando el warning `GStreamer: pipeline have not been created`.

2. Análisis de la Causa Raíz

El diagnóstico reveló que el problema principal no era de software de alto nivel, sino de la capa de transporte. ROS 2 utiliza DDS (*Data Distribution Service*) como middleware de comunicación. DDS fragmenta internamente los mensajes que superan el MTU (*Maximum Transmission Unit*) de la red. Un fotograma en formato crudo de 320×240 píxeles en espacio de color RGB24 ocupa:

$$320 \times 240 \times 3 \text{ bytes} = 230,400 \text{ bytes} \approx 225 \text{ KB por fotograma}$$

Esta cantidad es aproximadamente 160 veces mayor que el MTU estándar de Ethernet (1,500 bytes). DDS debe fragmentar cada fotograma en ~ 154 paquetes UDP. En una red Wi-Fi congestionada de laboratorio, la pérdida de **un solo fragmento** de los 154 provoca el descarte completo del fotograma por parte del receptor DDS, resultando en una tasa efectiva de recepción cercana a cero.

3. Solución Implementada: Pipeline UDP Personalizado

Se diseñó e implementó una solución que reemplaza el mecanismo de transporte de ROS 2 para la cámara por un pipeline de comunicación UDP personalizado en Python. La arquitectura se compone de dos módulos independientes:

3.1. Justificación del Protocolo: UDP frente a TCP

La elección del protocolo de transporte fue una decisión de diseño crítica. La tabla ?? compara los dos candidatos:

Característica	TCP	UDP
Establecimiento de conexión	Three-way handshake (3 RTT)	Inmediato (0 RTT)
Confirmación de entrega	Sí (ACK por cada paquete)	No
Retransmisión en pérdida	Sí (bloquea el emisor)	No
Control de flujo	Sí (ventana deslizante)	No
Overhead por paquete	20–60 bytes (cabecera TCP)	8 bytes (cabecera UDP)
Latencia para streaming	Alta (espera ACKs)	Baja

Cuadro 1: Comparativa TCP vs UDP para streaming de video en tiempo real

TCP fue descartado por dos razones fundamentales. Primero, el establecimiento de cada conexión TCP requiere completar el **three-way handshake**: el cliente envía un segmento **SYN**, el servidor responde con **SYN-ACK**, y el cliente confirma con **ACK**. Este proceso, aunque rápido en redes cableadas (< 1 ms), introduce latencia adicional impredecible en redes Wi-Fi donde el retardo de ida y vuelta (RTT) puede superar los 20–50 ms.

Segundo, y más crítico para video en tiempo real: cuando TCP detecta la pérdida de un paquete, **suspende la transmisión de todos los paquetes posteriores** hasta recibir la confirmación de reenvío del paquete perdido (mecanismo *head-of-line blocking*). En streaming de video, un fotograma perdido ya no tiene valor cuando llega con 100–500 ms de retraso, pues las demás partes del sistema han procesado varios fotogramas más recientes.

UDP, al ser un protocolo **no orientado a conexión y sin estado**, envía cada datagrama de forma independiente e inmediata. Si un fotograma se pierde en tránsito, el receptor simplemente descarta ese fotograma y procesa el siguiente, manteniendo la fluidez del stream. Esta filosofía de “frame reciente o nada” es exactamente el comportamiento correcto para un sistema de detección visual en tiempo real como YOLOv8 sobre un robot móvil.

3.2. Módulo Transmisor (udp_camera_sender.py) — Raspberry Pi

Ejecutado directamente en la Raspberry Pi del TurtleBot3 (sin depender de ROS 2), este script realiza las siguientes operaciones en un bucle continuo:

1. Captura un fotograma de `/dev/video0` usando OpenCV con el backend V4L2 explícito (`cv2.CAP_V4L2`) para evitar el conflicto con GStreamer.
2. Comprime el fotograma al formato JPEG con calidad configurable (80 %) mediante `cv2.imencode`. La compresión JPEG reduce el tamaño de un fotograma de 640×480 desde ~ 900 KB en crudo hasta $\sim 15\text{--}30$ KB, una reducción de más de 30:1.
3. Empaqueta los bytes JPEG con un encabezado de 4 bytes que indica el tamaño total del payload (estructura big-endian).
4. Envía el paquete completo mediante un único datagrama UDP a la IP de la laptop.

```
encode_params = [cv2.IMWRITE_JPEG_QUALITY, JPEG_QUALITY]
_, buffer = cv2.imencode('.jpg', frame, encode_params)
jpg_bytes = buffer.tobytes()
header = struct.pack('>I', len(jpg_bytes))
sock.sendto(header + jpg_bytes, (LAPTOP_IP, PORT))
```

Listing 1: Codificación y envío UDP del fotograma

La clave del funcionamiento es la compresión JPEG: al mantener cada datagrama por debajo de 65,507 bytes (límite teórico de UDP), se evita la fragmentación a nivel IP y el mensaje llega íntegro con un único datagrama.

3.3. Módulo Receptor (udp_camera_receiver_ros.py) — Laptop

Ejecutado en el entorno de ROS 2 de la laptop, este nodo actúa como un *bridge* entre el socket UDP y el ecosistema de ROS 2:

1. Abre un socket UDP y escucha en el puerto 5600.
2. Al recibir un datagrama, lee el encabezado de 4 bytes para validar la integridad del mensaje.
3. Decodifica los bytes JPEG de vuelta a una matriz NumPy mediante `cv2.imdecode`.
4. Convierte la matriz OpenCV al formato de mensaje de imagen de ROS 2 (`sensor_msgs/Image`) mediante `cv_bridge`.
5. Publica el mensaje en el tópico estándar `/camera/image_raw`, haciendo completamente transparente el cambio de transporte para el `DetectorNode` (YOLO) y demás consumidores.

Para garantizar que el nodo de YOLO siempre procese el fotograma más reciente sin acumular una cola de mensajes atrasados, la publicación al tópico se realiza desde un hilo separado con acceso mutuamente exclusivo (`threading.Lock`) al último fotograma recibido.

3.4. Diagrama del Flujo de Datos

```
[Camara USB /dev/video0]
  | V4L2 + cv2.VideoCapture
  v
[Raspberry Pi: udp_camera_sender.py]
  | Compresion JPEG 80% (~20 KB/frame)
  | UDP Socket puerto 5600 -- LAN
  v
[Laptop: udp_camera_receiver_ros.py]
  | Decodificacion JPEG -> Matriz BGR
  | cv_bridge
  v
[Topico ROS 2: /camera/image_raw]
  |
  +--> [DetectorNode: YOLOv8] --> /detected_objects --> [BrainNode]
  |
  +--> [rqt_image_view] (visualizacion)
```

4. Resultados Obtenidos

La implementación del pipeline UDP personalizado resultó en una mejora radical en todos los indicadores del sistema:

Métrica	Con usb_cam (ROS DDS)	Con Pipeline UDP
Resolución máxima estable	160 × 120 px	640 × 480 px
Framerate efectivo	< 1 FPS	15 FPS
Estabilidad de conexión	Desconexiones constantes	Estable durante toda la sesión
Detección de botella	No confiable	Funcional
Latencia estimada	Variable (pérdidas DDS)	< 200 ms

Cuadro 2: Comparativa de rendimiento antes y después de la solución

El sistema logró recibir y procesar fotogramas de forma continua a 15 FPS en resolución 640 × 480, verificado por el log del nodo receptor que reportaba 30 frames cada 2 segundos de forma consistente.

4.1. Límites de la Resolución: Restricciones de Hardware de la Raspberry Pi 4

Durante las pruebas se intentó elevar la resolución de transmisión a 1280 × 720 a 60 FPS con el objetivo de mejorar la precisión de YOLOv8. Este ajuste resultó impracticable por las limitaciones computacionales de la Raspberry Pi 4 operando en condiciones de carga múltiple.

La Raspberry Pi 4 (4 GB RAM, CPU Cortex-A72 de 4 núcleos a 1.8 GHz) ejecuta simultáneamente los siguientes procesos durante la operación del robot:

- **Sistema operativo Ubuntu 22.04** y daemons del kernel.

- **Stack de ROS 2 Humble** (`turtlebot3_bringup`): nodo de motores Dynamixel, LIDAR, odometría.
- **Nodo de cámara UDP** (`udp_camera_sender.py`): captura, codificación JPEG y transmisión de red.
- **Procesos de red Wi-Fi** para la interfaz `wlan1` dedicada a ROS 2.

La carga de codificar JPEG en tiempo real a 1280×720 píxeles por encima de 30 FPS satura los núcleos de la CPU, ya que OpenCV en ARM no dispone de aceleración hardware JPEG a través de V4L2 en esta configuración. El resultado observado fue un colapso de la tasa de transmisión: el script `udp_camera_sender.py` no lograba mantener el ritmo de captura, introduciendo latencia acumulada y caídas de frames. Adicionalmente, la contención de CPU provocaba que el nodo de motores perdiera sus ciclos de control Dynamixel, generando los errores `There is no status packet` incluso con batería cargada.

La resolución de 640×480 a 15 FPS representó el punto de equilibrio óptimo: suficiente para que YOLOv8 identifique objetos como botellas y teléfonos con confianza superior al 40 %, y compatible con la carga de CPU de la Raspberry Pi 4 en operación completa del robot.

5. Dificultades Adicionales y Soluciones

5.1. Desincronización de Relojes entre Raspberry Pi y Laptop

Dado que el router de la red de laboratorio no cuenta con salida a internet, la Raspberry Pi no pudo sincronizar su reloj NTP tras ser reiniciada. Esto generó una diferencia temporal de varios días entre la laptop y el robot, causando que Nav2 descartara todos los mensajes TF con el error:

```
Transform data too old: Data time: 1783377788s,
                        Transform time: 1761086572s
```

La solución fue sincronizar el reloj de la Raspberry Pi con el de la laptop mediante SSH:

```
ssh -t ubuntu@192.168.10.217 "sudo date -s \"$(date +'%Y-%m-%d_%T')\""
```

Listing 2: Sincronización del reloj vía SSH

5.2. Bajo Voltaje de la Batería (Brown-out)

Durante las pruebas, la batería del TurtleBot3 descendió por debajo de 11.1V, causando que la placa controladora OpenCR se desconectara del bus USB interno y generara un crash del proceso `turtlebot3_node` con el error `There is no status packet`. Este problema es inherente al hardware y se mitiga cargando la batería por encima del 80 % antes de cada sesión.

5.3. Exploración Autónoma sin Navegación por Waypoints

Los waypoints de búsqueda estaban calibrados para el entorno de simulación (casa virtual de Gazebo) y no correspondían a puntos válidos del mapa del laboratorio. Como solución, se rediseñó el **BrainNode** con un algoritmo de exploración aleatoria reactiva basado en el LIDAR. El robot avanza en línea recta, detecta obstáculos a menos de 40 cm, y gira un ángulo aleatorio entre 90° y 180° en dirección aleatoria para continuar la exploración, eliminando la dependencia de coordenadas de mapa codificadas manualmente.

6. Conclusiones

La Actividad 2 demostró que la implementación física de un sistema robótico supone desafíos fundamentalmente distintos a la simulación. Los tres hallazgos más importantes son:

1. **El middleware de ROS 2 (DDS) no es adecuado para streaming de video en redes Wi-Fi sin configuración explícita del transporte de imágenes comprimidas.** El pipeline UDP personalizado resultó ser una solución más simple, confiable y de mayor rendimiento, demostrando que a veces la ingeniería directa supera a las abstracciones del middleware.
2. **La sincronización temporal es un requisito crítico en sistemas distribuidos ROS 2.** La ausencia de NTP en redes aisladas debe anticiparse e incluirse en el procedimiento estándar de arranque del sistema físico.
3. **Los algoritmos de exploración reactiva son más robustos para entornos físicos** que no coinciden exactamente con la geometría del mapa simulado. La dependencia en waypoints codificados a mano introduce una fragilidad significativa al cambiar de entorno.